



PSDL LAB Manual 1 TO 5

information technology (Zeal College of Engineering and Research)



Scan to open on Studocu

Programming Skill Development Lab (214455)

LABORATORY MANUAL

Group A: Assignment No 1

Problem Statement:

Study of Embedded C programming language (Overview, syntax, One simple program like addition of two numbers).

Objective:

To understand the basic concepts of Embedded C and to write a simple code using it.

Outcome:

To understand a simple code of addition between two numbers.

CO Relevance: CO1**PO/PSOs Relevance:** PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3**Theory Concepts:**

1. Data types: Embedded C supports various data types such as integers, characters, floating-point numbers, and arrays. These data types are used to define variables and manipulate data within the program.
2. Control structures: Embedded C includes control structures like loops (for, while, do-while) and conditional statements (if-else, switch-case) to control the flow of program execution based on certain conditions.
3. Functions: Functions in Embedded C allow you to divide your code into smaller modules, making it easier to understand, debug, and maintain. Functions are defined with a return type, function name, input parameters, and a body containing the actual code.
4. Pointers: Pointers are variables that store memory addresses. They are extensively used in Embedded C for efficient memory management and accessing hardware registers and peripheral devices.
5. Bit manipulation: Embedded systems often deal with individual bits in hardware registers or perform bitwise operations. Embedded C provides bitwise operators (AND, OR, XOR, etc.) and bit manipulation techniques to work with individual bits.
6. Macros: Macros are preprocessor directives that allow you to define constants, inline functions, or perform text substitution before the actual compilation of the code. Macros are commonly used in Embedded C for defining register addresses and manipulating hardware-specific configurations.
7. Interrupts: Interrupts are a fundamental concept in embedded systems. They allow the system to respond to external events or hardware signals asynchronously. Embedded C provides mechanisms to handle interrupts efficiently using interrupt service routines (ISRs) and interrupt handling techniques.

Addition of two numbers:

Addition is one of the fundamental arithmetic operations and is used to combine or sum two or more numbers together to obtain a total or a sum.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pic18f4550.h>

void main(void) {
    int sum;
    sum=0;
    sum=0x03+0x04;
    TRISD=0;
    PORTD=sum;
}
```

Result:

The screenshot displays the source code for `add2numbers.c` in a microcontroller IDE. The code includes headers for `stdio.h`, `stdlib.h`, and `pic18f4550.h`. The `main` function initializes `sum` to 0, calculates `sum = 0x03 + 0x04`, sets `TRISD = 0`, and outputs the result to `PORTD`.

Below the code editor, the **RegisterTrace** window is open, showing the state of various registers. The registers listed are:

Address	Name	Hex	Decimal	Binary	Char
F84	PORTE	0x00	0	00000000	'.'
F89	LATA	0x00	0	00000000	'.'
F8A	LATB	0x00	0	00000000	'.'
F8B	LATC	0x00	0	00000000	'.'
F8C	LATD	0x07	7	00000111	'.'
F8D	LATE	0x00	0	00000000	'.'
F92	TRISA	0x7F	127	01111111	'Q'
F93	TRISB	0xFF	255	11111111	'y'
F94	TRISC	0xC7	199	11000111	'Ç'
F95	TRISD	0x00	0	00000000	'.'

Conclusion:

The addition of two numbers is verified successfully.

Group A: Assignment No 2

Problem Statement:

Write an Embedded C program to add array of n numbers.

Objective:

To implement Embedded C program to add array of n numbers.

Outcome:

To implement Embedded C program to add array of n numbers.

CO Relevance: CO1, CO2

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

To add an array of n numbers in an Embedded C program:

1. Declare an array of appropriate size and data type to store the numbers.
2. Initialize the array elements with desired values using assignments or a loop to read input.
3. Declare and initialize a variable to store the sum of the numbers.
4. Use a loop to iterate through the array, adding each element to the sum.
5. Utilize the sum for further processing or display it as the final result.

Program:

```
#include <stdio.h>
#include <stdlib.h>
#include <pic18f4550.h>

void main(void) {
    int i,sum,n;
    int number[]={ 1,2,3,4,5,6,7,8,9,10};
    sum=0;
    for(i=0;i<=9;i++){
        sum=sum+number[i];
    }
    TRISB=0;
    PORTB=sum;
}
```

Result:

The screenshot shows a C program in an IDE. The program calculates the sum of an array of numbers from 1 to 10. Below the code, a hardware register trace is displayed, showing the values of various registers after execution.

```

4  *
5  * Created on 12 March, 2023, 2:48 PM
6  */
7
8  #include <stdio.h>
9  #include <stdlib.h>
10 #include <pic18f4550.h>
11
12 void main(void) {
13     int i,sum,n;
14     int number[]={1,2,3,4,5,6,7,8,9,10};
15     sum=0;
16     for(i=0;i<=9;i++) {
17         sum=sum+number[i];
18     }
19     TRISB=0;
20     PORTB=sum;
21 }

```

RegisterTrace	Stimulus	Output	Notifications	Variables	Call Stack	Breakpoints
Address /	Name	Hex	Decimal	Binary	Char	
F84	PORTE	0x00	0	00000000	'.'	
F89	LATA	0x00	0	00000000	'.'	
F8A	LATB	0x37	55	00110111	'7'	
F8B	LATC	0x00	0	00000000	'.'	
F8C	LATD	0x00	0	00000000	'.'	
F8D	LATE	0x00	0	00000000	'.'	
F92	TRISA	0x7F	127	01111111	'O'	
F93	TRISB	0x00	0	00000000	'.'	
F94	TRISC	0xC7	199	11000111	'Ç'	
F95	TRISD	0xFF	255	11111111	'ÿ'	

Conclusion:

The addition of array of numbers is verified successfully.

Group A: Assignment No 3

Problem Statement:

Write an Embedded C program to transfer elements from one location to another for following:

- i) Internal to internal memory transfer
- ii) Internal to external memory transfer

Objective:

To understand the concept of data transfer from one location to location.

Outcome:

To implement the concept of transfer elements from one location to another for following:

- i) Internal to internal memory transfer
- ii) Internal to external memory transfer

CO Relevance: CO2

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

i) **Internal to Internal Memory Transfer:** In embedded systems, transferring elements from one internal memory location to another typically involves using pointer manipulation and memory copying techniques. The basic steps involved are:

1. Declare and initialize pointers for the source and destination memory locations.
2. Use memory copy functions like `memcpy()` or implement a custom copy algorithm to transfer the elements.
3. Specify the size of the data elements and the number of elements to be transferred.
4. Perform the memory transfer operation by copying the elements from the source location to the destination location.
5. Optionally, verify the transferred data for integrity or perform any required post-processing.

ii) **Internal to External Memory Transfer:** Transferring elements from internal memory to external memory in embedded systems typically involves interacting with peripheral interfaces (such as DMA controllers or memory mapped I/O) and following specific protocols defined by the external memory interface. The steps involved can vary depending on the specific hardware and interfaces used, but generally include:

1. Initialize the external memory interface and configure the appropriate settings for data transfer.
2. Allocate a buffer in the internal memory to store the data to be transferred.

3. Use memory copy functions or custom algorithms to copy the elements from the source internal memory to the allocated buffer.
4. Set up the external memory interface to initiate the transfer, providing the buffer address and the number of elements to be transferred.
5. Monitor the transfer status or interrupt signals, if applicable, to ensure the successful completion of the transfer. Optionally, perform error checking or recovery mechanisms.

Program:**Internal to Internal Transfer:**

```
#include <pic18f4550.h>

void main(void) {
    int temp,i;
    int source[]={0x21,0x22,0x23,0x24,0x25};
    int dest[]={0x00,0x00,0x00,0x00,0x00};
    for(i=0;i<=4;i++){
        dest[i]=source[i];
    }
}
```

Internal to External Transfer:

```
#include <pic18f4550.h>

void main(void) {
    int temp,i;
    int source[]={0x21,0x22,0x23,0x24,0x25};
    int dest[]={0x99,0x99,0x99,0x99,0x99};
    for(i=0;i<=4;i++)
    {
        temp=source[i];
        source[i]=dest[i];
        dest[i]=temp;
    }
    return;
}
```

Result:**Internal to Internal Transfer:**


```

7
8
9  #include <pic18f4550.h>
10
11 void main(void) {
12     int temp,i;
13     int source[]={0x21,0x22,0x23,0x24,0x25};
14     int dest[]={0x00,0x00,0x00,0x00,0x00};
15     for(i=0;i<=4;i++){
16         dest[i]=source[i];
17     }
18 }
19

```

main

RegisterTrace	Stimulus	Output	Notifications	SFRs	File Registers												
Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000	00	21	00	22	00	23	00	24	00	25	00	21	00	22	00	23	!. ". #. \$. %. !. ". #
010	00	24	00	25	00	05	00	00	00	00	00	00	00	00	00	00	. \$. %.
020	00	21	00	22	00	23	00	24	00	25	00	00	00	00	00	00	!. ". #. \$. %.
030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Internal to External Transfer:

```

8
9  #include <pic18f4550.h>
10
11 void main(void) {
12     int temp,i;
13     int source[]={0x21,0x22,0x23,0x24,0x25};
14     int dest[]={0x99,0x99,0x99,0x99,0x99};
15     for(i=0;i<=4;i++){
16     {
17         temp=source[i];
18         source[i]=dest[i];
19         dest[i]=temp;
20     }
21     }
22     return;
23 }

```

main

RegisterTrace	Stimulus	Output	Notifications	Variables	Call Stack	Breakpoints	SFRs	File Register									
Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000	00	99	00	99	00	99	00	24	00	25	00	21	00	22	00	23	 \$. %. !. ". #
010	00	24	00	99	00	24	00	04	00	99	00	99	00	99	00	99	. \$. %.
020	00	99	00	21	00	22	00	23	00	24	00	25	00	00	00	00	... !. ". #. \$. %.
030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Conclusion:

The transfer of elements from internal to internal and internal to external is verified successfully.

Group A: Assignment No 4

Problem Statement:

Write an Embedded C menu driven program for:

- i) Multiply 8-bit number by 8-bit number
- ii) Divide 8-bit number by 8-bit number

Objective:

To write an Embedded C program for multiply and divide 8-bit number by 8-bit number.

Outcome:

To implement the concept of transfer elements from one location to another for following:

- i) Internal to internal memory transfer
- ii) Internal to external memory transfer

CO Relevance: CO2

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

Multiplication:

1. Multiply two 8-bit numbers using the "unsigned" data type to ensure correct results.
2. Perform the multiplication using the "*" operator in Embedded C.
3. If the result exceeds 8 bits, the most significant bits will be truncated, and the result will wrap around within the 8-bit range.
4. Handle any overflow or carry conditions as needed, depending on the application requirements.

Division:

1. Divide an 8-bit number by another 8-bit number using the "unsigned" data type.
2. Perform the division using the "/" operator in Embedded C.
3. If the division is not exact, the result will be truncated to the nearest integer value.
4. Take into account any potential division by zero errors and handle them to avoid program crashes or undefined behavior.

Program:**Multiply:**

```
#include <pic18f4550.h>

void main(void) {
    int num1,num2;
    int result;
    result=0;
    num1=0x02;
    num2=0x04;
    result=num1 * num2;
    TRISB=0;
    PORTB=result;
    return;
} Divide:
```

```
#include <pic18f4550.h>

void main(void) {
    int num1,num2;
    int result;
    result=0;
    num1=0x08;
    num2=0x04;
    result=num1 / num2;
    TRISA=0;
    PORTA=result;
    return;
}
```

Result:**Multiply:**

```

8
9  #include <pic18f4550.h>
10
11 void main(void) {
12     int num1,num2;
13     int result;
14     result=0;
15     num1=0x02;
16     num2=0x04;
17     result=num1 * num2;
18     TRISB=0;
19     PORTB=result;
20     return;
21 }
22

```

main

RegisterTrace	Stimulus	Output	Notifications	SFRs x	
Address /	Name	Hex	Decimal	Binary	Char
F84	PORTE	0x00	0	00000000	'.'
F89	LATA	0x00	0	00000000	'.'
F8A	LATB	0x08	8	00001000	'.'
F8B	LATC	0x00	0	00000000	'.'
F8C	LATD	0x00	0	00000000	'.'
F8D	LATE	0x00	0	00000000	'.'
F92	TRISA	0x7F	127	01111111	'□'
F93	TRISB	0x00	0	00000000	'.'
F94	TRISC	0xC7	199	11000111	'Ç'
F95	TRISD	0xFF	255	11111111	'ÿ'

Divide:

```

8
9  #include <pic18f4550.h>
10
11 void main(void) {
12     int num1,num2;
13     int result;
14     result=0;
15     num1=0x08;
16     num2=0x04;
17     result=num1 / num2;
18     TRISA=0;
19     PORTA=result;
20     return;
21 }
22

```

main

RegisterTrace	Stimulus	Output	Notifications	Variables	Call Stack
Address /	Name	Hex	Decimal	Binary	Char
F84	PORTE	0x00	0	00000000	'.'
F89	LATA	0x02	2	00000010	'.'
F8A	LATB	0x00	0	00000000	'.'
F8B	LATC	0x00	0	00000000	'.'
F8C	LATD	0x00	0	00000000	'.'
F8D	LATE	0x00	0	00000000	'.'
F92	TRISA	0x00	0	00000000	'.'
F93	TRISB	0xFF	255	11111111	'ÿ'
F94	TRISC	0xC7	199	11000111	'Ç'
F95	TRISD	0xFF	255	11111111	'ÿ'

Conclusion:

The program for multiplying and dividing 8-bit number by 8-bit number is verified successfully.

Group A: Assignment No 5

Problem Statement:

Write an Embedded C program for sorting the numbers in ascending and descending order.

Objective:

To write an Embedded C program for sorting the numbers in ascending and descending order.

Outcome:

To understand sorting the numbers.

CO Relevance: CO2

PO/PSOs Relevance: PO1, PO2, PO3, PO4, PO5, PO9, PO10, PSO1, PSO2, PSO3

Theory Concepts:

➤ Bubble Sort:

- In bubble sort, adjacent elements are compared and swapped if they are in the wrong order, iterating through the list multiple times until the entire list is sorted.
- On each pass, the largest (or smallest) element "bubbles" to the end (or beginning) of the list.
- Bubble sort has a time complexity of $O(n^2)$, making it suitable for small lists but inefficient for large datasets.

Ascending Order:

1. To sort numbers in ascending order, you can use either bubble sort or quick sort algorithms.
2. Start with the first element and compare it with the next element, swapping them if they are in the wrong order. Continue this process until the largest element moves to the end.
3. Repeat the above step for the remaining elements, excluding the last one.
4. Iterate through the list until all elements are in their correct positions.

Descending Order:

1. To sort numbers in descending order, the same bubble sort or quick sort algorithms can be used.
2. Start with the first element and compare it with the next element, swapping them if they are in the wrong order. Continue this process until the smallest element moves to the end.
3. Repeat the above step for the remaining elements, excluding the last one.
4. Iterate through the list until all elements are in their correct positions.

Program:

```
#include <pic18f4550.h>

#define ARRAY_SIZE 5

void bubble_sort(int arr[], int size);

void main(void)
{
    int arr[ARRAY_SIZE] = {4, 2, 9, 6, 7};
    int i;

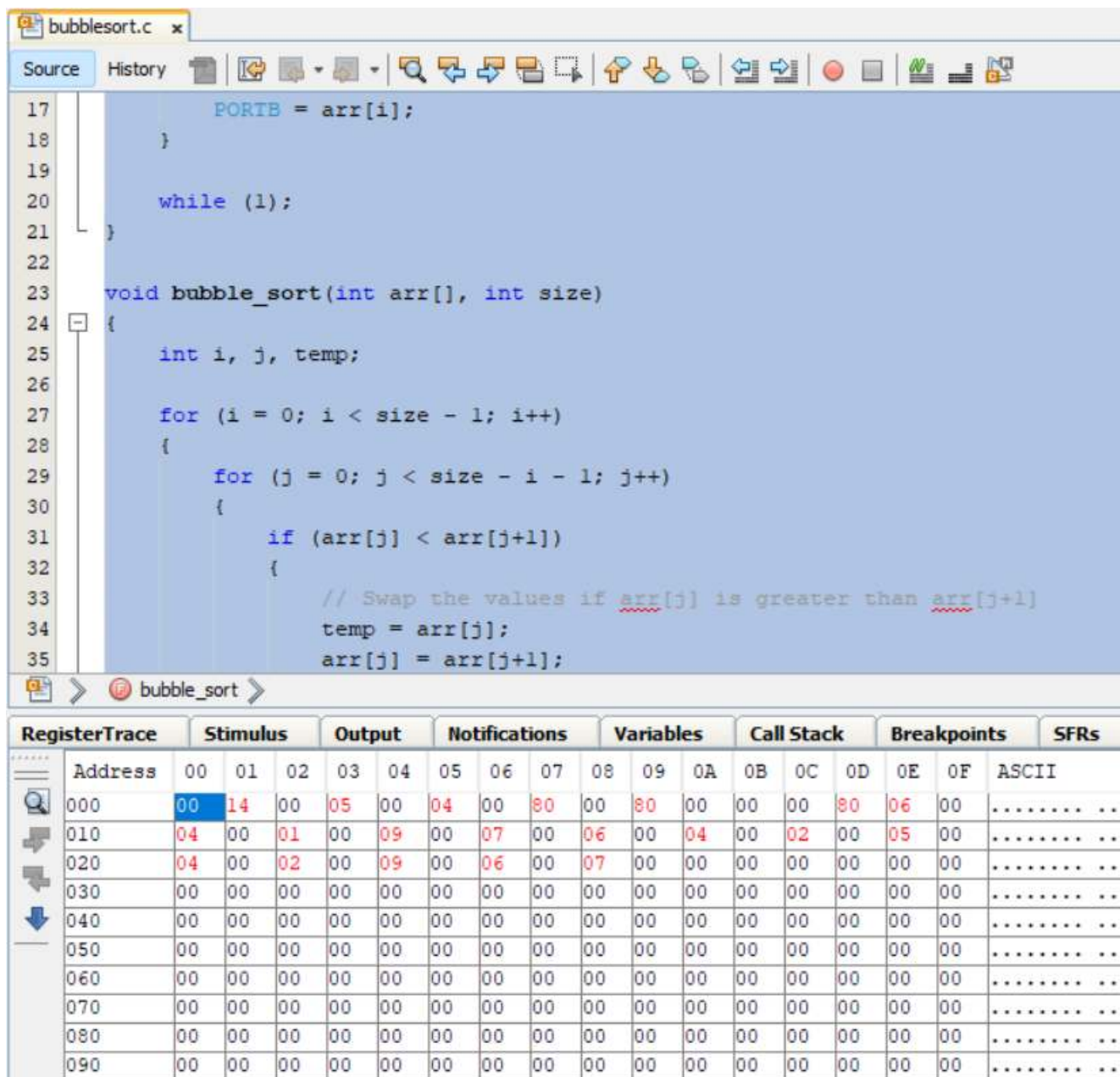
    bubble_sort(arr, ARRAY_SIZE);

    for (i = 0; i < ARRAY_SIZE; i++)
    {
        // Display sorted array values on PORTB
        PORTB = arr[i];
    }

    while (1);
}

void bubble_sort(int arr[], int size)
{
    int i, j, temp;

    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (arr[j] > arr[j+1])
            {
                // Swap the values if arr[j] is greater than arr[j+1]
                temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
}
```


Result:


```

17     PORTB = arr[i];
18 }
19
20 while (1);
21 }
22
23 void bubble_sort(int arr[], int size)
24 {
25     int i, j, temp;
26
27     for (i = 0; i < size - 1; i++)
28     {
29         for (j = 0; j < size - i - 1; j++)
30         {
31             if (arr[j] < arr[j+1])
32             {
33                 // Swap the values if arr[j] is greater than arr[j+1]
34                 temp = arr[j];
35                 arr[j] = arr[j+1];

```

RegisterTrace	Stimulus	Output	Notifications	Variables	Call Stack	Breakpoints	SFRs										
Address	00	01	02	03	04	05	06	07	08	09	0A	0B	0C	0D	0E	0F	ASCII
000	00	14	00	05	00	04	00	80	00	80	00	00	00	80	06	00	
010	04	00	01	00	09	00	07	00	06	00	04	00	02	00	05	00	
020	04	00	02	00	09	00	06	00	07	00	00	00	00	00	00	00	
030	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
040	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
050	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
060	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
070	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
080	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	
090	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	

Conclusion:

The program of sorting is verified successfully.